

CODE / IMPLEMENTATION FILE

Import Data using Transform Maps (Spreadsheet)

ServiceNow Micro Project

Important Note

The supplied project document describes a configuration-based ServiceNow implementation and does not contain custom source-code files. Therefore, the code in this separate file is supplementary sample ServiceNow JavaScript written to support the documented Employee Import → Transform Map → Employee Test workflow. It should not be presented as code copied from the original project document.

1. PROJECT CONFIGURATION

Item	Project Value
Source	Employee Import (u_employee_import)
Target	Employee Test (u_employee_test)
Transform Map	Sample Spreadsheet Import
Coalesce Field	Employee ID
Source Fields	Employee ID, Employee Name, Email, Department, Location
Target Fields	Employee ID, Employee Name, Email, Department, Location

2. TRANSFORM MAP – onBefore SCRIPT

Purpose: validate the Employee ID before a row is transformed. In a real ServiceNow instance, add this script to the Sample Spreadsheet Import Transform Map under the appropriate Transform Script configuration.

```
// Transform Map: Sample Spreadsheet Import
// Script type: onBefore

(function runTransformScript(source, target, map, log, isUpdate) {

    // Check whether Employee ID is available
    if (gs.nil(source.u_employee_id)) {
        ignore = true;
        log.error('Employee ID is missing. Import row ignored.');
```

```
        return;
    }

    // Trim imported employee ID
    source.u_employee_id = source.u_employee_id.toString().trim();

    // Keep the source value available for the target mapping
    target.u_employee_id = source.u_employee_id;

})(source, target, map, log, action);
```

3. TRANSFORM MAP – onAfter SCRIPT

Purpose: write an informational log after each Employee Test record has been processed.

```
// Transform Map: Sample Spreadsheet Import
// Script type: onAfter

(function runTransformScript(source, target, map, log, isUpdate) {

    if (action == 'insert') {
        log.info('Employee inserted: ' + target.u_employee_id);
    } else if (action == 'update') {
        log.info('Employee updated: ' + target.u_employee_id);
    } else {
        log.info('Employee transform action: ' + action);
    }

})(source, target, map, log, action);
```

4. DATA VALIDATION SCRIPT

This optional Background Script can be used by an administrator to inspect Employee Test records after a transform. It is a validation example and is not required for the documented project workflow.

```
// ServiceNow Background Script - validation example

var employee = new GlideRecord('u_employee_test');
employee.orderBy('u_employee_id');
employee.query();

while (employee.next()) {
    gs.info(
        'Employee ID: ' + employee.getValue('u_employee_id') +
        ' | Name: ' + employee.getValue('u_employee_name') +
        ' | Email: ' + employee.getValue('u_email') +
        ' | Department: ' + employee.getValue('u_department') +
        ' | Location: ' + employee.getValue('u_location')
    );
}
```

5. CHECK FOR AN EXISTING EMPLOYEE

The project uses Employee ID as the Coalesce field. The following example demonstrates the same matching idea with GlideRecord. The actual duplicate/update behaviour in the project is handled by Transform Map Coalesce configuration.

```
// Example: find an Employee Test record by Employee ID

var employeeId = 'SB-0001';

var gr = new GlideRecord('u_employee_test');
gr.addQuery('u_employee_id', employeeId);
gr.query();

if (gr.next()) {
    gs.info('Existing employee found: ' + gr.getValue('u_employee_name'));
} else {
    gs.info('No existing employee found for ID: ' + employeeId);
}
```

6. REPORT / DASHBOARD IMPLEMENTATION NOTES

The original project creates three reports rather than coding them. Their configuration is reproduced here for the implementation file:

- **Employees by Department** — Table: Employee Test; Type: Pie Chart; Group By: Department; Aggregation: Count.
- **Employees by Location** — Table: Employee Test; Type: Bar Chart; Group By: Location; Aggregation: Count.
- **Employee List Report** — Table: Employee Test; Type: List; Columns: Employee ID, Employee Name, Email, Department, Location.
- **Dashboard** — Employee Analytics Dashboards; the three reports are added as dashboard widgets.

7. EXPECTED TRANSFORM LOGIC

```
Spreadsheet
|
v
Employee Import (u_employee_import)
|
v
Sample Spreadsheet Import
|
+--> Employee ID used for Coalesce matching
|
v
Employee Test (u_employee_test)
|
+--> New Employee ID -> Insert
|
+--> Existing Employee ID + changed data -> Update
|
+--> Same data imported again -> No new duplicate record
|
v
Reports
|
v
Employee Analytics Dashboards
```

8. CONCLUSION

This code/implementation file provides sample ServiceNow JavaScript that complements the documented Transform Map project. The main project itself is configuration-driven: the Import Set table, Transform Map, field mappings, Coalesce setting, reports, and dashboard are created through ServiceNow configuration. The scripts above are optional examples for validation, logging, and controlled transformation behaviour.